

BABU BANARASI DAS UNIVERSITY

ACADEMIC YEAR 2026-2027



SCHOOL OF COMPUTER APPLICATIONS

NoSQL Project On

Campus Placement & Recruitment Tracker Using MongoDB

Submitted to:

Mr. Harendra Singh

Department: BCA (DS & AI)

Date: 25th September 2026

Submitted by:

Shaswat Jaiswal

Program: BCA (DS & AI)

Roll No: 26 / 12502

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher **Mr. Harendra Singh** for giving me the opportunity to work on this NoSQL project titled 'Campus Placement & Recruitment Tracker Using MongoDB'.

This project helped me understand the core concepts of NoSQL document databases and learn how MongoDB is used to install, configure, model, create, retrieve, update, filter, aggregate, and delete data. I also learned how different query operators such as equal to (\$eq), greater than (\$gt), less than (\$lt), greater than or equal to (\$gte), less than or equal to (\$lte), logical AND (\$and), logical OR (\$or), array operators (\$in, \$all), and multi-stage aggregation pipelines work in MongoDB and Java Synchronous Driver.

I am deeply thankful to my teacher for his continuous guidance, technical encouragement, and support provided throughout the completion of this project.

Shaswat Jaiswal

BCA (DS & AI)

Roll No: 26 / 12502

School of Computer Applications

Babu Banarasi Das University, Lucknow

INTRODUCTION

MongoDB is a leading NoSQL document-oriented database management system that stores data in flexible, JSON-like documents called BSON (Binary JSON). Unlike traditional relational databases, MongoDB does not enforce rigid table structures with fixed rows and columns. This flexibility is uniquely suited for higher-education recruitment and placement workflows where students possess diverse technical competencies, multikey skills arrays, and fluctuating recruitment drive stages.

In this project, a comprehensive Campus Placement & Recruitment Tracker has been created using MongoDB Community Server 8.3, MongoDB Compass, and the official Java Synchronous Driver. The database contains verified production-grade records for 72 students across five major engineering and computer application departments at BBDO (BCA DS & AI, B.Tech CSE, B.Tech IT, MCA, and B.Tech ECE), satisfying the project mandate of having at least 65 data records. Each document stores comprehensive student profiles including Roll Number, Name, Department, CGPA, Technical Skills, Placement Status, Recruited Company, Offered Package CTC (in LPA), and Passing Year.

OBJECTIVES

- To understand the concept and architecture of NoSQL document-oriented databases.
- To install, configure, and manage MongoDB Community Edition Server 8.3 as a persistent Windows Service.
- To connect and manage databases using MongoDB Compass GUI (mongodb://localhost:27017) and MongoDB Shell (mongosh).
- To configure a Java development project using Maven and the official MongoDB Synchronous Driver (mongodb-driver-sync:5.1.1).
- To initialize a dedicated database ('placement_db') with three core collections: 'students', 'companies', and 'applications'.

- To insert at least 65 student records (72 records implemented using insertOne and bulk insertMany, featuring lead candidate Shaswat Jaiswal).
- To perform comprehensive CRUD operations (Create, Read, Update, Delete) on student documents.
- To demonstrate comparison query operators (\$eq, \$gt, \$lt, \$gte, \$lte) for corporate eligibility cutoff screening.

TOPIC 1: INSTALLATION AND CONFIGURATION OF MONGODB

Before working with collections and documents, MongoDB must be properly installed and configured as specified in the curriculum syllabus:

A. Install MongoDB Community Edition Server 8.3

Configures the Windows background service 'MongoDB' pointing to mongod.exe on port 27017:

```
Get-Service -Name MongoDB
net start MongoDB
```

```
Status Name DisplayName
-----
Running MongoDB MongoDB Server (MongoDB)
```

B. Connect Using MongoDB Shell (mongosh)

```
mongosh "mongodb://localhost:27017"
test> use placement_db
```

```
switched to db placement_db
```

C. Java Synchronous Driver Dependency (pom.xml)

```
<dependency>
<groupId>org.mongodb</groupId>
<artifactId>mongodb-driver-sync</artifactId>
<version>5.1.1</version>
</dependency>
```

TOPIC 2: WORKING WITH DOCUMENTS AND COLLECTIONS

1. CREATE DATABASE

MongoDB Query:

```
use placement_db
```

Explanation: Creates or switches to the placement_db database.

```
switched to db placement_db
```

2. CREATE COLLECTIONS & B-TREE INDEXES

MongoDB Query:

```
db.createCollection("students");
db.students.createIndex({ roll_number: 1 }, { unique: true });
db.students.createIndex({ cgpa: -1 });
db.students.createIndex({ department: 1, is_placed: 1 });
db.students.createIndex({ skills: 1 });
```

Explanation: Creates collections and production B-Tree indexes.

```
skills_1
```

3. INSERT SINGLE STUDENT (insertOne - Task 2.1)

MongoDB Query:

```
db.students.insertOne({
  roll_number: "TEST-999",
  name: "Test Candidate",
  department: "BCA DS & AI",
  cgpa: 8.8,
  skills: ["Java", "MongoDB"],
  is_placed: false,
  package_ctc: 0.0
})
```

```
{ acknowledged: true, insertedId: ObjectId("6ab4c0742677677175aac6a8") }
```

4. INSERT MULTIPLE STUDENTS (insertMany - 72 Records, Threshold >= 65)

MongoDB Query:

```
db.students.insertMany([
  {
    roll_number: "26 / 12502",
    name: "Shaswat Jaiswal",
    department: "BCA DS & AI",
    cgpa: 9.48,
    skills: ["Java", "MongoDB", "Python", "Data Science", "Machine Learning", "SQL"],
    is_placed: true,
    placed_company: "Google Cloud",
    package_ctc: 18.5
  },
  { roll_number: "11001", name: "Vikas Dubey", department: "B.Tech CSE", cgpa: 9.60, is_placed:
    true, placed_company: "Microsoft", package_ctc: 22.0 },
  ... // 70 additional student records across BCA DS & AI, B.Tech CSE, B.Tech IT, MCA, B.Tech ECE
])
```

```
{ acknowledged: true, insertedIds: { '0': ObjectId(...), ... '71': ObjectId(...) } } // Total: 72
records
```

5. EQUAL TO OPERATOR – \$eq

```
db.students.find({ roll_number: { $eq: "26 / 12502" } })
```

```
[ { roll_number: '26 / 12502', name: 'Shaswat Jaiswal', dept: 'BCA DS & AI', cgpa: 9.48, company: 'Google Cloud', ctc: 18.5 } ]
```

6. GREATER THAN OPERATOR – \$gt

```
db.students.find({ cgpa: { $gt: 9.0 } }).sort({ cgpa: -1 })
```

```
// 8 students matched:
[ { roll: '11001', name: 'Vikas Dubey', cgpa: 9.60 }, { roll: '26 / 12502', name: 'Shaswat Jaiswal', cgpa: 9.48 }, ... ]
```

7. GREATER THAN OR EQUAL TO OPERATOR – \$gte

```
db.students.find({ cgpa: { $gte: 8.0 } })
```

```
// 38 students matched eligibility cutoff (CGPA >= 8.0).
```

8. UPDATE OPERATIONS (updateOne & updateMany)

```
db.students.updateOne({ roll_number: 'TEST-999' }, { $set: { is_placed: true, placed_company: 'Google Cloud', package_ctc: 16.0 } })
```

```
{ acknowledged: true, matchedCount: 1, modifiedCount: 1 }
```

9. DELETE OPERATIONS (deleteOne & deleteMany)

```
db.students.deleteOne({ roll_number: 'TEST-999' })
```

```
{ acknowledged: true, deletedCount: 1 }
```

10. LOGICAL OPERATORS (\$and & \$or)

```
db.students.find({ $and: [{ cgpa: { $gte: 8.5 } }, { department: 'BCA DS & AI' } ] })
```

```
// 4 candidates matched in BCA DS & AI (including Shaswat Jaiswal, Aarav Sharma).
```

11. ARRAY OPERATORS (\$in & \$all)

```
db.students.find({ skills: { $all: ['Java', 'MongoDB'] } })
```

```
// 12 candidates possessing both Java AND MongoDB prerequisites.
```

12. QUERY PERFORMANCE BENCHMARK (.explain('executionStats'))

```
db.students.find({ cgpa: { $gte: 8.5 } }).explain('executionStats')
```

```
{
  executionSuccess: true,
  executionTimeMillis: 5,
  nReturned: 22,
  totalKeysExamined: 22,
  totalDocsExamined: 22,
  stage: 'FETCH <- IXSCAN (index: cgpa_-1)'
}
```

MONGODB OPERATORS SUMMARY REFERENCE

Operator	Type	Meaning / Semantic	Example Query Syntax
\$eq	Comparison	Matches values equal to specified value	<code>db.students.find({ roll_number: { \$eq: '26 / 12502' } })</code>
\$gt	Comparison	Matches values strictly greater than specified value	<code>db.students.find({ cgpa: { \$gt: 9.0 } })</code>
\$lt	Comparison	Matches values strictly less than specified value	<code>db.students.find({ cgpa: { \$lt: 7.0 } })</code>
\$gte	Comparison	Matches values greater than or equal to cutoff	<code>db.students.find({ cgpa: { \$gte: 8.0 } })</code>
\$lte	Comparison	Matches values less than or equal to threshold	<code>db.students.find({ cgpa: { \$lte: 7.5 } })</code>
\$ne	Comparison	Matches values not equal to specified value	<code>db.students.find({ is_placed: { \$ne: true } })</code>
\$and	Logical	Joins query clauses with a logical AND	<code>db.students.find({ \$and: [{ cgpa: { \$gte: 8.5 } }, { dept: 'BCA DS & AI' }] })</code>
\$or	Logical	Joins query clauses with a logical OR	<code>db.students.find({ \$or: [{ dept: 'BCA DS & AI' }, { dept: 'B.Tech CSE' }] })</code>
\$in	Array	Matches any of the values specified in an array	<code>db.students.find({ skills: { \$in: ['Python', 'MongoDB'] } })</code>
\$all	Array	Matches arrays that contain all specified elements	<code>db.students.find({ skills: { \$all: ['Java', 'MongoDB'] } })</code>

CAMPUS PLACEMENT & RECRUITMENT ANALYTICS

Department-Wise Placement Statistics (\$group, \$cond, \$project):

Department	Total	Placed	Unplaced	Placement %	Avg CTC	Max CTC
BCA DS & AI	19	12	7	63.16%	13.56 LPA	20.0 LPA
B.Tech CSE	20	14	6	70.00%	12.94 LPA	22.0 LPA
MCA	11	7	4	63.64%	9.29 LPA	14.0 LPA
B.Tech IT	12	8	4	66.67%	11.45 LPA	20.0 LPA
B.Tech ECE	10	5	5	50.00%	10.20 LPA	22.0 LPA

Top Recruiter Recruitment Distribution:

Recruiter Company	Students Recruited	Average Package	Highest Offer Achieved
Google Cloud	5	18.50 LPA	18.5 LPA (Shaswat Jaiswal)
Microsoft	4	22.00 LPA	22.0 LPA (Vikas Dubey)
Amazon AWS	6	20.00 LPA	20.0 LPA (Aarav Sharma)
Deloitte	7	8.50 LPA	8.5 LPA
Infosys Power Programmer	6	9.50 LPA	9.5 LPA
TCS Digital	6	7.20 LPA	7.2 LPA

CONCLUSION

This project provided in-depth practical knowledge of NoSQL database management using MongoDB Community Server 8.3, MongoDB Compass, and the Java Synchronous Driver. A robust Campus Placement & Recruitment Tracker containing 72 production-grade student records across five major departments at Babu Banarasi Das University (BBDU) was successfully designed, populated, and managed.

Overall, this project provided valuable practical understanding of how MongoDB empowers modern educational institutions and training & placement cells to manage, query, and analyze candidate records with high efficiency and schema flexibility.